GUIA TÉCNICO

Preparação de Dados, Engenharia de Atributos e Pipelines

Propósito — Material didático autoral para revisão e estudo independente. Base: notebook correspondente e conceitos atualizados do scikit-learn.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. Conceitos e hipóteses
2. Fundamentos matemáticos
3. Experimento reproduzível
4. Interpretação e limitações
5. Exercícios orientados
6. Boas práticas de validação

01 | Contexto e decisões fundamentais

Imputação, codificação, escalonamento e seleção devem ser ajustados dentro da validação. Pipeline e ColumnTransformer encapsulam esse comportamento e reduzem inconsistência entre treino e inferência. Variáveis categóricas precisam de política explícita para níveis inéditos; os valores ausentes podem ter significado informativo.

Ponto de atenção

Fazer fit_transform em toda a tabela antes de cross-validation permite vazamento de estatísticas globais. Campos identificadores não são automaticamente bons atributos preditivos.

02 | Fundamentação e hipóteses

1. Fundamentos, objetivos e contexto

Objetivos. Tratar ausências, categorias, escalas e engenharia de atributos sem vazamento.

ColumnTransformer aplica tratamentos por coluna; Pipeline conecta transformações a modelo. Colunas categóricas podem conter valores novos no teste, então OneHotEncoder(handle_unknown="ignore") é mais robusto do que mapear categorias manualmente. Seleção de atributos e reamostragem, quando existem, também devem ocorrer apenas dentro dos folds de treino.

Interpretação prática

Imputação, codificação, escalonamento e seleção devem ser ajustados dentro da validação. Pipeline e ColumnTransformer encapsulam esse comportamento e reduzem inconsistência entre treino e inferência. Variáveis categóricas precisam de política explícita para níveis inéditos; os valores ausentes podem ter significado informativo.

2. Fundamentos matemáticos

Padronização: z=(x-mu_{mathrm{treino}})/sigma_{mathrm{treino}}. O uso de mu ou sigma estimados sobre todos os dados vaza informações do teste. Para variáveis categóricas, uma representação *one-hot* é um vetor indicador por categoria conhecida.

Conexão com disciplinas anteriores: revise os conceitos matemáticos e estatísticos correspondentes. Não repetimos integralmente os capítulos de Python, visualização e inferência estatística.

3. Experimento guiado: preparando a pergunta

Antes do código, identifique: (a) quais dados são de treino, (b) qual informação poderá entrar no modelo, (c) que métrica ou propriedade será observada, e (d) que conclusão não é permitida pelos resultados. Estes exemplos são didáticos, com semente fixa onde aplicável.

Exemplo reproduzível selecionado

```
import pandas as pd
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

EXEMPLO COMPLETO NO APÊNDICE

O código integral está no apêndice deste guia, em formato copiável e sem linhas truncadas.

Como interpretar

Os números são resultados específicos do experimento e da amostra, não garantias universais. Verifique o que mudou, qual método foi usado e qual é o cenário de aplicação. Compare com a teoria matemática e não confunda desempenho de treino, validação e teste.

4. Segundo experimento ou extensão controlada

Mude apenas uma condição principal (algoritmo, parâmetro ou tipo de dado), quando possível, e registre o efeito. Não ajuste decisões com base no teste final.

5. Exercício resolvido

Enunciado: reforce o conceito central com um exemplo menor, cujo resultado pode ser inspecionado diretamente.

6. Interpretação, limitações e erros comuns

Não confundir imputação com recuperação do valor verdadeiro. *Encoding* ordinal só é adequado quando as categorias realmente possuem ordem, ou quando o algoritmo lida corretamente com a codificação. Pipelines melhoram reprodutibilidade e segurança metodológica; não garantem, por si sós, ausência de vazamento na construção original das colunas.

Perguntas de revisão:
• Qual hipótese implícita sustenta o procedimento usado?
• Que erro de implementação ou de validação poderia tornar o resultado artificialmente otimista?
• Como você mudaria o experimento para refletir dados reais?

Risco metodológico

Fazer fit_transform em toda a tabela antes de cross-validation permite vazamento de estatísticas globais. Campos identificadores não são automaticamente bons atributos preditivos.

7. Consulta rápida

• Fazer split ANTES de fit/fit_transform de qualquer transformação aprendida.
• Pipeline dentro de cross_val_score reestima transformadores a cada fold.
• Usar ColumnTransformer para dados heterogêneos.
• Validar valores desconhecidos e colunas ausentes.

Prática orientada e critérios de avaliação

Exercício A — sensibilidade: altere um hiperparâmetro/hipótese e apresente uma tabela pequena com resultados e explicação. Exercício B — revisão crítica: construa um exemplo de uso incorreto (sem aplicá-lo a dados reais) e explique o procedimento correto. Exercício C — comunicação: escreva um parágrafo sobre escopo, métricas, suposições e limitações do método.

Critério de aprovação: códigos executam do começo ao fim, não há seleção pelo teste final e cada resultado possui interpretação.

Aprofundamento v2 — auditoria e aplicação

Ponto de atenção deste capítulo: pipeline dentro do treino. Este complemento não substitui as seções e exemplos anteriores.

Verificação conceitual. Explique, com suas palavras, em qual etapa do fluxo de trabalho o tema pipeline dentro do treino pode produzir conclusões enganosas. Indique um indicador observável, um teste mínimo e uma limitação do procedimento.

Exercício de extensão. Reproduza o experimento central deste notebook alterando um parâmetro por vez. Compare com uma linha de base simples, reporte a variação da métrica pertinente e justifique o resultado. Quando houver treino/teste, qualquer etapa que aprenda parâmetros deve usar apenas o treino.

Checklist de conclusão: (1) entradas identificadas; (2) sementes registradas; (3) saídas interpretadas; (4) premissas explicitadas; (5) dados originais não sobrescritos.

Laboratório adicional — Pré-processamento sem vazamento

Experimento independente do estado das células anteriores. Use-o como verificação e explique seus limites; resultados em conjuntos sintéticos não são estimativas de desempenho real.

Exercício resolvido — protocolo mínimo de validação

Enunciado. Proponha uma comparação para pipeline dentro do treino sem contaminar a avaliação.

Solução orientadora. Defina uma hipótese falsificável, escolha um baseline simples, fixe uma divisão adequada ao problema, altere uma única decisão metodológica e compare métricas no conjunto reservado. Não transforme exemplos sintéticos em conclusões sobre populações reais. Documente a configuração e os casos de falha. Quando o tema não envolver predição, substitua o baseline por uma implementação de referência e compare correção, cobertura e custo.

Síntese de verificação

Verifique se a divisão dos dados representa a aplicação, se pré-processamento e seleção foram ajustados apenas no treino e se as métricas são compatíveis com o custo de erro. Registre sementes, versões e limitações. Nenhuma pontuação obtida em amostra didática garante desempenho em outro contexto.

Documentação técnica

scikit-learn: https://scikit-learn.org/stable/ ; NumPy: https://numpy.org/doc/ ; SciPy: https://docs.scipy.org/doc/scipy/ ; Jupyter: https://jupyter.org/ .

Portfólio GitHub: https://github.com/eduardojr-pereira/data-science-portfolio

Site pessoal: https://eduardo-pereira.webflow.io/

APÊNDICE — CÓDIGO REPRODUZÍVEL (1/1)

Fonte: 04_preparacao_pipelines.ipynb

```
import pandas as pd
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder,StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import balanced_accuracy_score
rng=np.random.default_rng(42);n=180
X=pd.DataFrame({'idade':rng.normal(42,12,n),'renda':rng.normal(3000,900,n),
    'canal':rng.choice(['online','loja'],n)})
X.loc[rng.choice(n,18,replace=False),'renda']=np.nan
y=((X['idade']>45) & (X['canal']=='online')).astype(int)
Xtr,Xte,ytr,yte=train_test_split(X,y,stratify=y,test_size=.25,random_state=42)
num=Pipeline([('imputer',SimpleImputer(strategy='median')),('scale',
    StandardScaler())])
cat=Pipeline([('imputer',SimpleImputer(strategy='most_frequent')),('onehot',
    OneHotEncoder(handle_unknown='ignore'))])
prep=ColumnTransformer([('numericas',num,['idade','renda']),('categoricas',cat,
    ['canal'])])
model=Pipeline([('prep',prep),('classificador',
    LogisticRegression(max_iter=1000))]).fit(Xtr,ytr)
print('Acurácia balanceada teste:',round(balanced_accuracy_score(yte,
    model.predict(Xte)),3))
```